

DATABRICKS CONFIDENTIAL

Zerobus Direct Write API: Private Preview Onboarding Doc

Last Updated: Oct 1, 2025

→ Review the [prerequisites](#) before getting started

Support and feedback

Feedback: Book a meeting with the Product Management team | [Booking Link](#)

General questions: Please email victoria.bukta@databricks.com with feedback, questions, and requests!

Table Of Contents

[Support and feedback](#)

[Overview](#)

[How does it work?](#)

[Prerequisites](#)

[Limitations](#)

[User Guide](#)

[Appendix](#)

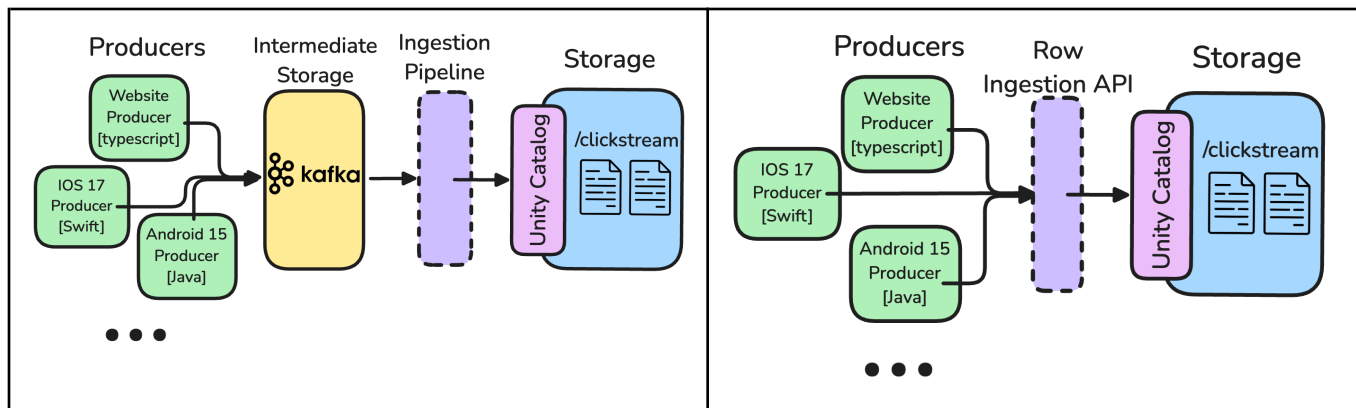
Overview

Databricks is previewing a robust API that allows customers to efficiently push data into tables with ease. It supports record-by-record ingestion at any scale and operates in a serverless environment.

This API is designed to accommodate a variety of ingestion use cases, particularly benefiting clients who may face challenges when integrating with systems or writing directly in Delta Lake format. As long as clients can communicate via [gRPC](#) and construct [Protobuf](#) messages, they can use this API.

Before

After



- In the example above we are able to streamline the ingestion of clickstream data.
- The message bus is removed from the architecture.
- Applications can build integration against a standard API interface, removing the need for Delta or message bus-specific dependencies.

When data is transmitted to the Zerobus Ingest API, it goes through a buffering process before being added to a Delta table. This creates an efficient and durable ingestion mechanism to support a high volume of clients with variable throughput.

Once the data has been materialized into Delta format, it becomes fully compatible with the comprehensive Databricks Platform, allowing users to leverage familiar tools and functionalities for further data analysis and processing.

Concepts

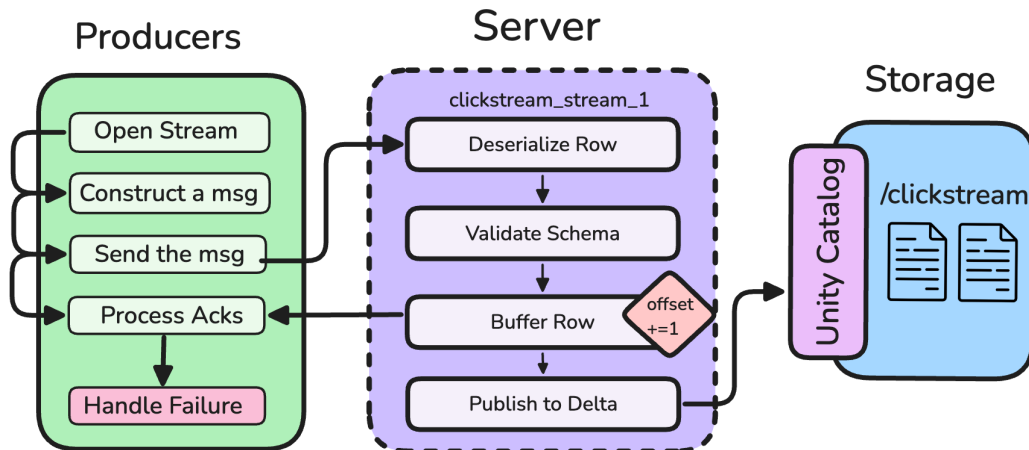
A data producer will first open a “stream” to a Delta table, construct a message matching its schema, and then push the message to our API. Our service will make the data durable, acknowledge the client’s message, and materialize in the Delta table.

Server

Our service does not create or manipulate tables automatically. It simply takes data from clients, validates that it fits into the table, and then writes the data to the table.

Our responsibility includes:

- Schema validation of the message to the table.
- Materializing the data in a timely manner to the target table.
- Sending an acknowledgement to the client that the data is durable.



Client

Client integration involves:

1. Selecting a target table.
2. Establishing a stream with our service.
3. Constructing a schema-compatible message.
4. Sending the message.
5. Managing message acknowledgements.
6. Implementing recovery mechanisms in the case of client, stream, or server-side failures (e.g., connection issues, schema mismatches)

Our [Python SDK](#), provides user-friendly methods to accomplish this, and our documentation offers examples of [different development patterns](#). For custom integrations, our SDK can serve as a reference for integration structure and recovery handling.

How does it work?

[Step 1] Get an Zerobus URL from Databricks

[Step 2] Create or identify the table you want to ingest data to

[Step 3] Issue PAT token privileges to the table.

[Step 4] Write a client to start sending data!

- Check out our ["User Guide"](#) for details

Prerequisites

- **Confirmation of the Private Preview Terms of Service**
- [Disable IP access list for the workspace.](#)

Limitations

Databricks is releasing this Private Preview at a very early stage. As with any Private Preview, it's not intended for production workloads. However, this preview has a particularly large set of limitations. Our goal is to learn early and quickly adapt the product to customers' requirements.

Networking	We cannot support workspaces configured with a customer-managed VPC.
Durability	We only support single availability zone (single-az) durability. This means our service may experience downtime if our zone is unavailable.
Throughput Restrictions	Same AZ • 100MB/second per stream • 15,000 rows per second per stream Note: Max throughput can be achieved when a client app and endpoint are within the same geo region. If you need higher throughput, please reach out to your Databricks team.
Delivery Guarantees	At-Least-Once Delivery: An acknowledgement from the server means that the record is durable and is in the Delta table.
Workspace	The workspace must be in one of the following regions: AWS: us-east-1, us-east-2, us-west-2, eu-central-1, ap-southeast-1, ap-southeast-2, ap-northeast-1, ca-central-1, eu-west-1, eu-central-1 Azure: us-east-2, West Europe
Databricks Notebooks	Currently, only notebooks running with classic compute are supported with our Python SDK.
Target Table	We support writing to Managed Delta tables only at this time. Note: We do not support writing to managed storage at this time.
	The table location must be in the same region as your workspace. The following regions are supported: AWS: us-east-1, us-east-2, us-west-2, eu-central-1, ap-southeast-1, ap-southeast-2, ap-northeast-1, ca-central-1, eu-west-1, eu-central-1 Azure: us-east-2, West Europe
	We do not support recreating a target table at this time.
	At this time, we only support table names with ASCII letters/digits/underscores.
Schema Evolution	We do not support schema evolutions on the target table at this time.

Protobuf Schema	At this time the protobuf schema definition must match 1:1 with the Delta table schema. If not, the API returns an error. This includes: - Number of columns - Column names - Column optionality must match between Delta and Protobuf schemas	
	At this time, we do not support proto schemas with more than 2000 columns.	
	At this time, we only support table and column names with ASCII letters/digits/underscores.	
	At this time, we do not support using a different proto schema for "stream creation" and "ingest record".	
Record Size	Each individual message is limited to 4MB in size at this time. Note: Record size max is 4,194,285 Bytes . Headers required for communication take up 19 Bytes.	
Type Support	Delta types	Protobuf types
	Integer	Int32
	String	String
	Float	Float
	Long	Int64
	Short	Int32
	Double	Double
	Boolean	Bool
	Binary	Bytes
	Date*	Int32
	Timestamp**	Int64
	Array<TYPE>	repeated TYPE
	Map<K, V>	map<K, V>
	Struct<FIELDS>	message Nested { FIELDS }

*Should be converted to int32 (**number of days** since epoch)

Should be converted to int64 (epoch time in **microseconds)

***The *map* Protobuf syntactic sugar is available only for Protobuf compilers version 3 and above

User Guide

Step 1: Get your workspace URL.

When viewing your Databricks workspace after logging in, take a look at the URL in your browser with the following format: `https://<databricks-instance>.com/o=XXXXX`. The URL that you require is everything before the `/o=XXXXX`

Example

Full URL=<https://abcd-teste2-test-spcse2.cloud.databricks.com/?o=2281745829657864#>

Workspace URL=<https://abcd-teste2-test-spcse2.cloud.databricks.com>

Step 2: Create or identify the target table you want to ingest data to.

```
// Create a table
CREATE TABLE unity.default.air_quality (
  device_name STRING, temp INT, humidity LONG
);
```

Step 3: Create a Service Principal and grant it permissions to the table.

A Service Principal is a specialized identity providing more security than personalized accounts. More information regarding Service Principal and how to use them for authentication can be found in [these instructions](#).

1. Create a new Service Principal in your workspace under Settings > Identity and Access.
2. Generate and save client ID and secret for that Service Principal.
3. Grant the required permissions for the catalog, schema and table to the newly created Service Principal:
 - a. For the catalog: **USE_CATALOG**.
 - b. For the schema: **USE_SCHEMA**.
 - c. For the table: **MODIFY, SELECT**.

Step 4: Write a client.

→ [Python SDK](#)

→ [Rust SDK](#)

→ [Java SDK](#)

→ [REST API](#)

Python SDK

Step 1: Install our Python SDK.

- Install the python wheel that was sent to you in your onboarding package.

```
> pip install databricks_zerobus-0.0.17-py3-none-any.whl
```

Step 2: Create a proto-message definition for your table.

- Generate proto files from the destination UnityCatalog table by using the `generate_proto` script that's available after installing the SDK

```
> generate_proto \  
  --uc-endpoint "<Workspace URL>" \  
  --uc-token "<PAT>" \  
  --table "<catalog.schema.table_name>" \  
  --proto-msg "<AirQuality>" \  
  --output "<my_filename>.proto"
```

- You should end up with something that looks like the following.
 - Note: The numbers indicate the column ordering and are not default values.
 - You can learn more about Protobuf messages [here](#).

```
syntax = "proto2";  
  
message AirQuality {  
  optional string device_name = 1;  
  optional int32 temp = 2;  
  optional int64 humidity = 3;  
}
```

Step 3: Compile the Protobuf message definition.

- You can compile the protobuf definition with `grpcio-tools`.
- Run the command below in the command line to compile a specified "proto" file.
- You will find a ".py" file in your local directory, which is the compiled python proto-definition.
- Include this file in your Python client to have the proto-definition to create records.

```
> pip install grpcio-tools  
> python3 -m grpc_tools.protoc -I. --python_out=. <my_filename>.proto
```

Step 4: Create a Python client.

To build a Python client, you will need to import:

1. Your Python compiled "proto" definition that you generated in [Step 3](#).
2. Our SDK.

To use the SDK you will need the following information:

- [Databricks Workspace URL](#)
- [Table Name](#)
- [Service Principal client ID and secret](#)
- Zerobus URI = [<workspace_id>.zerobus.<region>.cloud.databricks.com](#)

- Databricks will provide you with this URL.

Here is an example application.

→ Python SDK usage patterns can be found [here](#).

```
import asyncio
import row_pb2 # This is the compiled proto definition from step 3.
from zerobus_sdk.aio import ZerobusSdk
from zerobus_sdk import TableProperties, StreamConfigurationOptions, get_zerobus_token

SERVER_ENDPOINT = "<WORKSPACE_ID>.zerobus.<region>.cloud.databricks.com"
WORKSPACE_URL = "<WORKSPACE_URL>"
TABLE_NAME = "unity.default.air_quality"

table_properties = TableProperties(TABLE_NAME, row_pb2.AirQuality.DESRIPTOR)

sdk_handle = ZerobusSdk(SERVER_ENDPOINT)

CLIENT_ID = "<CLIENT_ID>"
CLIENT_SECRET = "<CLIENT_SECRET>"

async def example():
    # Create stream configuration options.
    options = StreamConfigurationOptions(
        token_factory=lambda: get_zerobus_token(
            TABLE_NAME, SERVER_ENDPOINT.split(".")[0], WORKSPACE_URL,
            CLIENT_ID, CLIENT_SECRET))

    # Create stream to table.
    stream = await sdk_handle.create_stream(table_properties, options)

    # non-blocking (streaming) call.
    NUM_RECORDS = 1_000_000
    for i in range(NUM_RECORDS):
        if ((i + 1) % 1000) == 0:
            print(">> Sent " + str(i + 1) + " records to ingest.")

        # we are awaiting it to be queued
        await stream.ingest_record(row_pb2.AirQuality(
            device_name="device_num_" + str(i),
            temp=i,
            humidity=i + 1
        ))

    # Wait until we receive the ack for the latest record
    await stream.flush()
```

```
# Close the stream
await stream.close()

if __name__ == "__main__":
    asyncio.run(example())
```

Rust SDK

Step 1: Download Rust SDK zip

- Download the SDK zip from the link that was sent to you
- Extract the zip
- You should see something like this

```
rust_sdk/
├── tools/      # These are helper files.
│   ├── build_proto.sh
│   ├── generate_proto.py
│   └── build_proto/
├── sdk/        # The SDK crate.
└── readme.md
```

Step 2: Make a new Rust project

- Create a new project with

```
> cargo new example_project
```

You should get something like

```
example_project/
├── Cargo.toml
├── src/
│   └── main.rs
```

Step 3: Place the contents of rust_sdk inside your rust project

- You should get this:

```
example_project/
├── Cargo.toml
├── src/
│   └── main.rs
├── sdk/
└── tools/
```

- In *Cargo.toml* add

```
[dependencies]
```

```
zerobus_sdk = { path = "./sdk" }
```

Step 4. Run the script inside the tools folder

- This script helps generate the proto definition for your table, along with a descriptor and the corresponding Rust file.
- *Note: You must have Python and Rust (with Cargo) installed.*

```
> cd example_project
> chmod +x tools/build_files.sh
> ./tools/build_files.sh \
  --uc-endpoint <WORKSPACE_URL> \
  --uc-token <PAT> \
  --table <CATALOG.SCHEMA.TABLE> \
  --output_file_name <PROTO_NAME>.proto \
  --output_dir output
```

- You should see a folder created with 3 files inside

```
example_project/
├── Cargo.toml
├── output/
│   ├── <something>.rs
│   ├── <something>.proto
│   └── <something>.descriptor
├── src/
│   └── main.rs
├── sdk/
└── tools/
```

Step 5. Create a client

To use the SDK you will need the following information:

- [Databricks Workspace URL](#)
- [Table Name](#)
- [PAT](#)
- Zerobus URI = [<workspace_id>.ingest.cloud.databricks.com](#)
 - Databricks will provide you with this URL.

Here is an example application.

- This is an example with a mock table "air_quality".
- After generating all of the files, our file tree looks like this:

```

example_project/
├── Cargo.toml
├── output/
│   ├── air_quality.rs
│   ├── air_quality.proto
│   └── air_quality.descriptor
├── src/
│   └── main.rs
├── sdk/
└── tools/

```

air_quality.proto

```

syntax = "proto2";

message AirQuality {
  optional string device_name =
1;
  optional int32 temp = 2;
  optional int64 humidity = 3;
}

```

air_quality.rs

```

// @generated by prost-build

#[derive(Clone, PartialEq, ::prost::Message)]
pub struct AirQuality {
  #[prost(string, optional, tag = "1")]
  pub device_name: Option<String>,
  #[prost(int32, optional, tag = "2")]
  pub temp: Option<i32>,
  #[prost(int64, optional, tag = "3")]
  pub humidity: Option<i64>,
}

```

- **Note: In this example, you need to add these dependencies to Cargo.toml for a successful run**

```

[dependencies]
zerobus_sdk = { path = "../sdk" }
tonic-build = "0.13.1"
prost = "0.13.3"
prost-build = "0.12"
prost-reflect = "0.14.2"
tokio = { version = "1.0", features = ["macros", "rt-multi-thread"] }
tokio-stream = "0.1.16"
tonic = { version = "0.12.3", features = ["tls"] }
tracing = "0.1.41"
chrono = "0.4"

```

- You run the program with

```

> cd example_project

```

```
> cargo run
```

```
Rust
use std::error::Error;
use zerobus_sdk::{ZerobusSdk, TableProperties};
use prost_reflect::{prost_types};
use std::fs;
use prost::Message;

// Change constants to match your data.
const DATABRICKS_WORKSPACE_URL: &str =
    "https://<YOUR_DATABRICKS_WORKSPACE_URL>";
const TABLE_NAME: &str = "unity.default.air_quality";
const PAT: &str = "<YOUR_PAT>";
const SERVER_ENDPOINT: &str =
    "https://<WORKSPACE_ID>.ingest.cloud.databricks.com";

pub mod air_quality {include!("../output/air_quality.rs");} // Module name is
arbitrary.
use crate::air_quality::AirQuality;

#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
    let descriptor_proto = load_descriptor_proto(
        "output/air_quality.descriptor", "air_quality.proto", "air_quality");
    let table_properties = TableProperties {
        table_name: TABLE_NAME.to_string(),
        descriptor_proto,
    };
    let sdk_handle = ZerobusSdk {
        zerobus_endpoint: SERVER_ENDPOINT.to_string(),
        unity_catalog_url: Some(DATABRICKS_WORKSPACE_URL.to_string()),
        token: Some(PAT.to_string()),
        use_tls: true,
```

```

};

let mut stream =
sdk_handle.create_stream(table_properties.clone(), None).await
    .expect("Failed to create a stream.");

let ack_future = stream.ingest_record(AirQuality {
    device_name: Some("sensor-001".to_string()),
    temp: Some(22),
    humidity: Some(55),
}).encode_to_vec()).await.unwrap();

let _ack = ack_future.await.unwrap();
let close_future = stream.close();
close_future.await?;
Ok(())
}

fn load_descriptor_proto(path: &str, file_name: &str, message_name: &str) ->
prost_types::DescriptorProto {
    let descriptor_bytes = fs::read(path).expect("Failed to read proto
descriptor file");
    let file_descriptor_set =
prost_types::FileDescriptorSet::decode(descriptor_bytes.as_ref()).unwrap();

    let file_descriptor_proto = file_descriptor_set.file.into_iter()
        .find(|f| f.name.as_ref().map(|n| n.as_str()) == Some(file_name))
        .unwrap();

    file_descriptor_proto.message_type.into_iter()
        .find(|m| m.name.as_ref().map(|n| n.as_str()) == Some(message_name))
        .unwrap()
}

```

Java SDK

Step 1: Prepare the project directory.

- Your project structure should look like this:

```
example_project/  
├─ jar/
```

Step 2 - Option 1: Download the Java SDK JAR.

- Download the Java SDK JAR that was sent to you in your onboarding package and place it in the `jar` folder. Your project structure should now look like this:

```
example_project/  
├─ jar/  
|   └─ databricks_zerobus-0.0.6.jar
```

Step 2 - Option 2: Download the Java SDK fat JAR.

- Download the Java SDK **fat** JAR that was sent to you in your onboarding package and place it in the `jar` folder. The uber JAR contains all required third party dependencies. Your project structure should now look like this:

```
example_project/  
├─ jar/  
|   └─ databricks_zerobus-0.0.6_fat.jar
```

Step 3 : Download required third party JARs. (Skip if fat JAR is used)

- **IMPORTANT:** The versions of the `protobuf-java` JAR and `protoc` used to compile the Protobuf file should be equal for maximum safety and support. If a version mismatch exists errors are possible during compilation or runtime of Java code.
- Download the `slf4j-api`, `slf4j-simple` and `protobuf-java` JARs and place them in the `jar` folder:

```
> cd jar/  
> wget https://repo1.maven.org/maven2/org/slf4j/slf4j-api/2.0.16/slf4j-api-2.0.16.jar  
> wget https://repo1.maven.org/maven2/org/slf4j/slf4j-simple/2.0.17/slf4j-simple-2.0.17.jar  
> wget https://repo1.maven.org/maven2/com/google/protobuf/protobuf-java/4.29.3/protobuf-java-4.29.3.jar
```

- Your project structure should now look like this:

```
example_project/  
├─ jar/  
|   └─ databricks_zerobus-0.0.6.jar  
|   └─ protobuf-java-4.29.3.jar
```

```
|   ├── slf4j-api-2.0.16.jar
|   ├── slf4j-simple-2.0.17.jar
```

Step 2 - Option 1: Generate a proto-message definition and Java file for your table.

- Both the regular and the uber JARs contain scripts for generating a Protobuf file together with a Java file for a destination Delta table. You can utilize this by running the following commands:

```
> cd example_project
> unzip jar/databricks_zerobus-0.0.6.jar 'scripts/*'
> chmod +x scripts/build_proto.sh
> scripts/build_proto.sh \
  --uc-endpoint <WORKSPACE_URL> \
  --uc-token <PAT> \
  --table <CATALOG.SCHEMA.TABLE> \
  --output-file-name record.proto
```

- Your project structure should now look like this:

```
example_project/
├── record.proto
├── RecordOuterClass.java
├── jar/
│   ├── databricks_zerobus-0.0.6.jar
│   ├── protobuf-java-4.29.3.jar
│   ├── slf4j-api-2.0.16.jar
│   └── slf4j-simple-2.0.17.jar
├── scripts/
│   ├── build_proto.sh
│   └── generate_proto.py
```

Step 2 - Option 2: Create a proto-message definition for your table.

- Create a Protobuf message that matches your Delta table and place it in the `example_project` folder.
 - Note: The numbers indicate the column ordering and are not default values.
 - You can learn more about Protobuf messages [here](#).

Protobuf

```
syntax = "proto2";
```

```
// Optionally declare the Java package and name for the generated Java Protobuf class.
```

```
// option java_package = "com.test";
```

```
// option java_outer_classname = "RecordOuterClass";

message AirQuality {
    optional string device_name = 1;
    optional int32 temp = 2;
    optional int64 humidity = 3;
}
```

- Your project structure should now look like this:

```
example_project/
├── record.proto
├── jar/
│   ├── databricks_zerobus-0.0.6.jar
│   ├── protobuf-java-4.29.3.jar
│   ├── slf4j-api-2.0.16.jar
│   └── slf4j-simple-2.0.17.jar
```

Step 3: Compile the Protobuf message definition. (Skip if script was used for generating)

- You can compile the Protobuf definition with `protoc`.
- Run the command below in the command line to compile a specified "proto" file.

```
> protoc -I=./ --java_out=./ ./record.proto
```

- Your project structure should now look like this:

```
example_project/
├── record.proto
├── RecordOuterClass.java
├── jar/
│   ├── databricks_zerobus-0.0.6.jar
│   ├── protobuf-java-4.29.3.jar
│   ├── slf4j-api-2.0.16.jar
│   └── slf4j-simple-2.0.17.jar
```

Step 4: Create a Java client.

- To use the SDK you will need the following information:
 - [Databricks Workspace URL](#)
 - [Table Name](#)
 - [PAT](#)
 - Zerobus URI = [<workspace_id>.ingest.cloud.databricks.com](#)
 - Databricks will provide you with this URL.

- Create a `Test.java` file and place it in the `example_project` folder. Below is an example application:

Java

```
import com.databricks.zerobus_sdk.*;
import com.databricks.zerobus.*;

import java.util.ArrayList;
import java.util.concurrent.CompletableFuture;
import java.util.function.Consumer;

public class Test {
    private static final String TABLE_NAME = "unity.catalog.air_quality";
    private static final String ZEROBUS_ENDPOINT = "<WORKSPACE_ID>.ingest.cloud.databricks.com";
    private static final String UNITY_CATALOG_ENDPOINT = "<YOUR_DATABRICKS_WORKSPACE_URL>";
    private static final String UC_TOKEN = "<YOUR_PAT>";

    private static final int NUM_RECORDS = 100_000;

    public static void main(String[] args) {
        try {
            // Callback that will be called when an acknowledgment is received from the server.
            Consumer<IngestRecordResponse> ackCallback = response ->
                System.out.println("Record ack for offset: " + response.getDurabilityAckUpToOffset());

            // Table properties including table name and record schema class.
            TableProperties<RecordOuterClass.AirQuality> tableProps =
                new TableProperties<>(TABLE_NAME, RecordOuterClass.AirQuality.class);

            // Stream configuration including max in-flight records and callback.
            StreamConfigurationOptions options = StreamConfigurationOptions.builder()
                .setMaxInflightRecords(50_000)
                .setAckCallback(ackCallback)
                .build();

            // Create the Zerobus SDK handle.
            ZerobusSdk zerobusSdk =
                new ZerobusSdk(ZEROBUS_ENDPOINT, UNITY_CATALOG_ENDPOINT, UC_TOKEN);

            // Create the Zerobus stream.
            ZerobusStream<RecordOuterClass.AirQuality> stream =
                zerobusSdk.createStream(tableProps, options);

            System.out.println("Stream created: " + stream.getStreamId());

            // Prepare and ingest records.
            System.out.println("Ingesting " + NUM_RECORDS + " records ...");

            ArrayList<CompletableFuture<?>> futures = new ArrayList<>(NUM_RECORDS);

            // Build a sample record (reused for all inserts).
            RecordOuterClass.AirQuality row = RecordOuterClass.AirQuality.newBuilder()
                .setDeviceName("device_1")
                .setTemp(20)
                .setHumidity(50)
                .build();
```

```

        long startTime = System.currentTimeMillis();

        for (int i = 0; i < NUM_RECORDS; i++) {
            IngestRecordResult result = stream.ingestRecord(row);

            // Blocking on the future that the SDK accepted the record.
            result.getRecordAccepted().get();

            // Not blocking on the future that the record was made durable.
            futures.add(result.getWriteCompleted());

            if ((i + 1) % 10_000 == 0) {
                System.out.println("Ingested " + (i + 1) + " records");
            }
        }

        // Wait for all records to be acknowledged.
        // NOTE: It is possible that callbacks are called after flush finishes.
        System.out.println("Flushing records ...");
        stream.flush();

        long endTime = System.currentTimeMillis();
        System.out.println("Flush completed, ingested " + NUM_RECORDS +
            " records in " + (endTime - startTime) + "ms");

        // Wait for all ingestion futures to complete.
        for (CompletableFuture<?> future : futures) {
            future.get();
        }

        // Close the stream.
        stream.close();
        System.out.println("Stream closed successfully.");
        System.out.println("Records per second: " +
            (NUM_RECORDS * 1000.0 / (endTime - startTime)));
    } catch (ZerobusException e) {
        System.err.println("Error creating stream: " + e.getMessage());
    } catch (Exception e) {
        System.err.println("Unexpected error: " + e.getMessage());
    }
}
}

```

- Your project structure should now look like this:

```

example_project/
├─ Test.java
├─ record.proto
├─ RecordOuterClass.java
├─ jar/
│   └─ databricks_zerobus-0.0.6.jar
│   └─ protobuf-java-4.29.3.jar

```

```
|    ├── slf4j-api-2.0.16.jar
|    ├── slf4j-simple-2.0.17.jar
```

Step 5: Compile and run the application.

- To compile the application, position your terminal in the `example_project` folder and run the following command:

```
> cd example_project/
> javac -d ./bin/ -cp "./jar/*" Test.java RecordOuterClass.java
```

- To run the application, position your terminal in the `example_project/bin` folder and run the following command:

```
> cd bin/
> java -cp "../jar/*" Test
```

- **IMPORTANT:** The `javac` version should not be higher than that of `java`. Protobuf is supported from Java 8.

REST API

The REST API allows you to ingest a single record by sending an HTTP POST request to the `/ingest-record` endpoint. The record itself is included in the request body and must be in JSON format.

For creating the request you will need the following information:

- Databricks Workspace URL
- Table Name
- PAT
- Zerobus URI = `<workspace_id>.ingest.cloud.databricks.com`
 - Databricks will provide you with this URL.

Request URL

`<workspace_id>.ingest.cloud.databricks.com/ingest-record?table_name=<table_name>`

The request URL consists of three parts:

- the Zerobus URI - `<workspace_id>.ingest.cloud.databricks.com`,
- the endpoint path - `/ingest-record`,
- the `table_name` query parameter with the full name of the target table consisting of the catalog, schema and table name (ex. `unity.default.air_quality`).

Headers

The request requires three specific HTTP headers to properly authenticate and format the request.

- Content-Type: application/json - Mandatory field for specifying the content type. Currently Json is the only supported message format.
- Authorization: Bearer <token> - Replace <token> with the PAT token you previously created.
- unity-catalog-endpoint: <workspace_url> - Replace <workspace_url> with your own URL.
- x-databricks-zeroconf-table-name: <table_name> - Used for routing. The value <table_name> has to match with the one used in the query parameter.

Request Body

The request body contains the data you want to ingest. It must be a single JSON object. The keys in the JSON object have to precisely match the column names in the target table.

Example Body:

```
JSON
{
  "device_name": "device_num_1",
  "temp": 28,
  "humidity": 60
}
```

Full example using curl

Here is a complete curl command targeting the unity.default.air_quality table, showing how all the components fit together. Remember to replace the placeholder values (<PAT> and <workspace_url>) with your actual credentials.

```
Shell
curl -X POST
"https://<WORKSPACE>.ingest.cloud.databricks.com/ingest-record?table_name=unity.default.air_quality" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <PAT>" \
-H "unity-catalog-endpoint: <workspace_url>" \
-H "x-databricks-zeroconf-table-name: unity.default.air_quality" \
-d '{
  "device_name": "device_num_1",
  "temp": 28,
  "humidity": 60,
}'
```

If everything is correct, you should receive an HTTP 200 response with the text "Successfully ingested record." in the request body.

Appendix

FAQ

How will I be charged for this service?

We will allow our early design partners to test free of charge.

Python SDK Patterns

Our Python SDK is available in two variations:

1. **Synchronous (sync)**: A traditional, blocking client that uses standard threading for concurrency. This is the default client.
2. **Asynchronous (async)**: A non-blocking client based on Python's [asyncio](#) library, suitable for high-throughput I/O-bound applications.

You can import them as follows:

Python

```
# Import the default synchronous SDK
from zerobus_sdk import (ZerobusSdk, TableProperties,
                          StreamConfigurationOptions,
                          get_zerobus_token)
```

Python

```
# To use the asynchronous SDK, you must import it from the .aio submodule
from zerobus_sdk.aio import ZerobusSdk
from zerobus_sdk import TableProperties, StreamConfigurationOptions,
get_zerobus_token
```

This document covers patterns for both clients.

Blocking Call

A "blocking call" in this context means your code will wait until a single record has been sent to the server and the server has acknowledged that the record is durable (i.e., successfully written).

Synchronous Client

With the synchronous client, you call `ingest_record` which returns a `RecordAcknowledgment` object. You can then call `.wait_for_ack()` on this object to block until the acknowledgment is received.

Python

```
import row_pb2
from zerobus_sdk import (ZerobusSdk, TableProperties,
                          StreamConfigurationOptions,
                          get_zerobus_token)

DATABRICK_WORKSPACE_URL = "<YOUR_DATABRICKS_WORKSPACE_URL>"
TABLE_NAME = "unity.default.air_quality"
CLIENT_ID = "<CLIENT_ID>"
CLIENT_SECRET = "<CLIENT_SECRET>"
SERVER_ENDPOINT = "<WORKSPACE_ID>.zerobus.<region>.cloud.databricks.com"

sdk_handle = ZerobusSdk(SERVER_ENDPOINT)

table_properties = TableProperties(TABLE_NAME, row_pb2.AirQuality.DESRIPTOR)

def blocking_call():
    # Create stream configuration options.
    options = StreamConfigurationOptions(
        token_factory=lambda: get_zerobus_token(
            TABLE_NAME, SERVER_ENDPOINT.split(".")[0], DATABRICK_WORKSPACE_URL,
            CLIENT_ID, CLIENT_SECRET))

    # Create stream to table.
    stream = sdk_handle.create_stream(table_properties, options)

    # Ingest a record. This call returns immediately.
    record_ack = stream.ingest_record(
        row_pb2.AirQuality(device_name="my_device", temp=17, humidity=42)
    )

    # Block and wait for the record to be acked and durable
    offset_id = record_ack.wait_for_ack()
```

```

    print(f"Record with ID: {offset_id} persisted.")

    stream.close()

if __name__ == "__main__":
    blocking_call()

```

Asynchronous Client

With the asynchronous client, `ingest_record` is an `async` method that returns an `asyncio.Future`. You can `await` this future to block until the record is durable.

```

Python
import asyncio
import row_pb2
from zerobus_sdk.aio import ZerobusSdk
from zerobus_sdk import TableProperties, StreamConfigurationOptions,
get_zerobus_token

DATABRICK_WORKSPACE_URL = "<YOUR_DATABRICKS_WORKSPACE_URL>"
TABLE_NAME = "unity.default.air_quality"
CLIENT_ID = "<CLIENT_ID>"
CLIENT_SECRET = "<CLIENT_SECRET>"
SERVER_ENDPOINT = "<WORKSPACE_ID>.zerobus.<region>.cloud.databricks.com"

# Note: For the async client, we import from zerobus_sdk.aio
sdk_handle = ZerobusSdk(SERVER_ENDPOINT)

table_properties = TableProperties(TABLE_NAME, row_pb2.AirQuality.DESRIPTOR)

async def blocking_call():
    # Create stream configuration options.
    options = StreamConfigurationOptions(
        token_factory=lambda: get_zerobus_token(
            TABLE_NAME, SERVER_ENDPOINT.split(".")[0], DATABRICK_WORKSPACE_URL,
            CLIENT_ID, CLIENT_SECRET))

    # Create stream to table.
    stream = await sdk_handle.create_stream(table_properties, options)

    # Wait for the record just to be queued
    record_ack_future = await stream.ingest_record(

```

```

        row_pb2.AirQuality(device_name="my_device", temp=17, humidity=42)
    )

    # Wait for the record to be acked and durable
    await record_ack_future

if __name__ == "__main__":
    asyncio.run(blocking_call())

```

Non-Blocking Call With Callback

For high-throughput applications, you can ingest records without waiting for each one to be acknowledged individually. This allows the SDK to buffer records and send them in the background. You can receive acknowledgments via callbacks.

Synchronous Client

The synchronous client can operate in a "non-blocking" mode by simply not calling `wait_for_ack()` on every record. You can use the `ack_callback` in `StreamConfigurationOptions` to receive batch acknowledgments, or add a per-record callback using `record_ack.add_done_callback()`.

```

Python
import row_pb2
from zerobus_sdk import (ZerobusSdk, TableProperties,
                          StreamConfigurationOptions,
                          get_zerobus_token)

DATABRICK_WORKSPACE_URL = "<YOUR_DATABRICKS_WORKSPACE_URL>"
TABLE_NAME = "unity.default.air_quality"
CLIENT_ID = "<CLIENT_ID>"
CLIENT_SECRET = "<CLIENT_SECRET>"
SERVER_ENDPOINT = "<WORKSPACE_ID>.zerobus.<region>.cloud.databricks.com"

sdk_handle = ZerobusSdk(SERVER_ENDPOINT)

table_properties = TableProperties(TABLE_NAME, row_pb2.AirQuality.DESRIPTOR)

def record_ack_callback(response):
    offset_id = response.durability_ack_up_to_offset

```

```

print(f"Received stream-level ack for offset {offset_id}")

def non_blocking_call_with_callback():
    # Open a stream with a callback function for batch acks.
    # Note: The callback function will not be called on every message offset.
    #       The server returns the largest acknowledged offset.
    stream_options = StreamConfigurationOptions(
        ack_callback=record_ack_callback,
        token_factory=lambda: get_zerobus_token(
            TABLE_NAME, SERVER_ENDPOINT.split(".")[0], DATABRICK_WORKSPACE_URL,
            CLIENT_ID, CLIENT_SECRET))
    stream = sdk_handle.create_stream(table_properties, stream_options)

    NUM_RECORDS = 1_000_000
    for i in range(NUM_RECORDS):
        if ((i + 1) % 1000) == 0:
            print(">> Sent " + str(i + 1) + " records to ingest.")

        # This call is non-blocking (unless the internal buffer is full)
        stream.ingest_record(row_pb2.AirQuality(
            device_name="device_num_" + str(i),
            temp=i,
            humidity=i + 1
        ))

    # Wait for all acks to be received.
    stream.flush()

    # Close the stream
    stream.close()

if __name__ == "__main__":
    non_blocking_call_with_callback()

```

Even though sending to the server and receiving acknowledgments is parallelized for all these records, `ingest_record` will block if the internal buffer is full (`max_inflight_records` in `StreamConfigurationOptions`).

Asynchronous Client

The async client achieves the same result using `asyncio`. You can `await stream.ingest_record(record)` which queues the record for sending but returns a future immediately. You don't have to `await` the returned future right away.

Python

```
import asyncio
import row_pb2
from zerobus_sdk.aio import ZerobusSdk
from zerobus_sdk import TableProperties, StreamConfigurationOptions,
get_zerobus_token

DATABRICK_WORKSPACE_URL = "<YOUR_DATABRICKS_WORKSPACE_URL>"
TABLE_NAME = "unity.default.air_quality"
CLIENT_ID = "<CLIENT_ID>"
CLIENT_SECRET = "<CLIENT_SECRET>"
SERVER_ENDPOINT = "<WORKSPACE_ID>.zerobus.<region>.cloud.databricks.com"

sdk_handle = ZerobusSdk(SERVER_ENDPOINT)

table_properties = TableProperties(TABLE_NAME, row_pb2.AirQuality.DESRIPTOR)

def record_ack_callback(response):
    offset_id = response.durability_ack_up_to_offset
    print(f"Received ack for {offset_id}")

async def example_with_callback():
    # Open a stream with a callback function
    stream_options = StreamConfigurationOptions(
        ack_callback=record_ack_callback,
        token_factory=lambda: get_zerobus_token(
            TABLE_NAME, SERVER_ENDPOINT.split(".")[0], DATABRICK_WORKSPACE_URL,
            CLIENT_ID, CLIENT_SECRET))
    stream = await sdk_handle.create_stream(table_properties, stream_options)

    NUM_RECORDS = 1_000_000
    for i in range(NUM_RECORDS):
        if ((i + 1) % 1000) == 0:
            print(">> Sent " + str(i + 1) + " records to ingest.")

        await stream.ingest_record(row_pb2.AirQuality(
            device_name="device_num_" + str(i),
            temp=i,
            humidity=i + 1
        ))

    # Wait for all acks to be received.
    await stream.flush()
```

```

# Close the stream
await stream.close()

if __name__ == "__main__":
    asyncio.run(example_with_callback())

```

Ingestion in parallel

To achieve maximum throughput, you may want to ingest data from multiple sources in parallel. The strategy for this depends on whether you are using the sync or async client.

Parallelism with the Synchronous Client

The synchronous SDK is thread-safe. You can share a single **ZerobusSdk** handle across multiple threads, but each thread must create its own **ZerobusStream** instance.

However, due to Python's Global Interpreter Lock (GIL), there is **no performance benefit** to using this multi-threaded approach. The SDK's internal stream handling already saturates the GIL, so creating additional threads will not result in a speedup.

To achieve true parallelism and better performance, you should use a process pool (e.g., Python's **ProcessPoolExecutor**). Although processes have a higher overhead than threads, they are not constrained by the GIL, providing a significant performance improvement.

We are currently investigating ways to overcome this limitation in the future to offer more efficient parallelism.

```

Python
from concurrent.futures import ProcessPoolExecutor, as_completed
import row_pb2
from zerobus_sdk import ZerobusSdk, TableProperties,
StreamConfigurationOptions, get_zerobus_token

# --- Constants ---
# These constants will be available to each worker process.
DATABRICKS_WORKSPACE_URL = "<YOUR_DATABRICKS_WORKSPACE_URL>"
TABLE_NAME = "unity.default.air_quality_managed"
CLIENT_ID = "<CLIENT_ID>"
CLIENT_SECRET = "<CLIENT_SECRET>"

```

```

SERVER_ENDPOINT = "<WORKSPACE_ID>.zerobus.<region>.cloud.databricks.com"

def ingest_records_blocking(task_id: int, num_records: int, ack_offset: int):
    """
    A single high-throughput task that sends records using the blocking SDK.
    Each process initializes its own SDK handle.
    """
    stream = None
    try:
        # Initialize sdk_handle within the worker process.
        # This is the recommended pattern for ProcessPoolExecutor to ensure
        # that each process has its own isolated connection/resource handle.
        sdk_handle = ZerobusSdk(SERVER_ENDPOINT)

        # Create stream to table. Each process creates its own stream.
        table_properties = TableProperties(TABLE_NAME,
row_pb2.AirQuality.DESRIPTOR)

        # Create stream configuration options with token factory
        options = StreamConfigurationOptions(
            token_factory=lambda: get_zerobus_token(
                TABLE_NAME, SERVER_ENDPOINT.split(".")[0],
DATABRICKS_WORKSPACE_URL,
                CLIENT_ID, CLIENT_SECRET))

        stream = sdk_handle.create_stream(table_properties, options)
        print(f"Task {task_id}: Stream created.")

        for i in range(num_records):
            record = row_pb2.AirQuality(device_name=f"device_{task_id}_{i}",
temp=i, humidity=i + 1)

            # Blocking call which completes when the message is persisted.
            record_future = stream.ingest_record(record)

            # Optionally wait for an acknowledgement (ack).
            # The ack_offset parameter controls how often to wait for an ack.
            if ((i + 1) % ack_offset) == 0 or i == num_records - 1:
                print(f"Task {task_id}: Sent {i + 1} records.")
                print(f"    -> Task {task_id}: Waiting for acknowledgement
(flush)...")
                result = record_future.wait_for_ack()

```

```

        print(f" -> Task {task_id}: Records up to offset {result} are
persisted.")

    except Exception as e:
        print(f"An error occurred in task {task_id}: {e}")
    finally:
        if stream:
            stream.close()
            print(f"Task {task_id}: Stream closed.")

if __name__ == "__main__":
    print("Starting parallel ingestion workload using ProcessPoolExecutor...")

    num_parallel_streams = 4
    records_total = 400_000
    ack_offset = 100_000

    # Calculate records per stream
    records_per_stream = records_total // num_parallel_streams

    # Use ProcessPoolExecutor to run tasks in parallel processes.
    with ProcessPoolExecutor(max_workers=num_parallel_streams) as executor:
        futures = [executor.submit(ingest_records_blocking, i,
records_per_stream, ack_offset)
                    for i in range(num_parallel_streams)]

        # Wait for all futures to complete.
        for future in as_completed(futures):
            try:
                future.result() # Re-raises any exception caught by the
function.
            except Exception as e:
                print(f"A process encountered an error: {e}")

    print(f"Total Records: {records_total:,}")
    print("Parallel ingestion workload completed.")

```

Note: When ingesting in parallel across multiple streams, there is no guarantee of the order in which records will be ingested across those streams. If you need to preserve order, you must ingest all ordered records through the same stream.

Asynchronous Client with `asyncio.gather`

The asynchronous SDK uses an event loop on a single thread. To achieve parallelism, you should create multiple `asyncio` tasks and run them concurrently using `asyncio.gather`. This is highly efficient for I/O-bound operations.

It is an **anti-pattern** to mix the `asyncio` client with traditional `threading`.

```
Python
import asyncio
import row_pb2
from zerobus_sdk.aio import ZerobusSdk
from zerobus_sdk import TableProperties, StreamConfigurationOptions,
get_zerobus_token

DATABRICK_WORKSPACE_URL = "<YOUR_DATABRICKS_WORKSPACE_URL>"
TABLE_NAME = "unity.default.air_quality"
CLIENT_ID = "<CLIENT_ID>"
CLIENT_SECRET = "<CLIENT_SECRET>"
SERVER_ENDPOINT = "<WORKSPACE_ID>.zerobus.<region>.cloud.databricks.com"

sdk_handle = ZerobusSdk(SERVER_ENDPOINT)
table_properties = TableProperties(TABLE_NAME, row_pb2.AirQuality.DESRIPTOR)

async def worker(worker_id: int, num_records: int):
    """An async worker that creates a stream and ingests records."""
    print(f"Worker {worker_id}: Starting...")

    # Create stream configuration options with token factory
    options = StreamConfigurationOptions(
        token_factory=lambda: get_zerobus_token(
            TABLE_NAME, SERVER_ENDPOINT.split(".")[0], DATABRICK_WORKSPACE_URL,
            CLIENT_ID, CLIENT_SECRET))

    stream = await sdk_handle.create_stream(table_properties, options)

    for i in range(num_records):
        await stream.ingest_record(row_pb2.AirQuality(
            device_name=f"device_{worker_id}_{i}", temp=i, humidity=i
        ))

    await stream.flush()
    await stream.close()
    print(f"Worker {worker_id}: Finished.")
```

```

async def run_parallel():
    num_workers = 10
    records_per_worker = 1000
    tasks = []
    for i in range(num_workers):
        task = asyncio.create_task(worker(i, records_per_worker))
        tasks.append(task)

    await asyncio.gather(*tasks)

if __name__ == "__main__":
    asyncio.run(run_parallel())

```

Recovery

By default, the Python SDK performs automatic recovery. When a stream fails (e.g., due to a transient network issue), the SDK will attempt to reconnect and re-ingest any unacknowledged records in the background, preserving their order. This behavior is identical for both the **sync** and **async** clients.

Recovery is controlled by the following **StreamConfigurationOptions**:

- **recovery** (bool, default: **True**): Enables or disables the auto-recovery feature. If **False**, a stream failure will immediately raise an **ZerobusException**. If the stream fails, the SDK will throw an *ZerobusException* and records that weren't acknowledged by the server will just be retrieved so the user can see what records failed to be ingested (and potentially do some logic).
- **recovery_retries** (int, default: **3**): The number of times the SDK will attempt to recover the stream. If SDK doesn't succeed in recovering the stream after this number of retries, SDK will throw *ZerobusException* (everything is the same as above)
- **recovery_timeout_ms** (int, default: **7000**): The timeout in milliseconds for a single recovery attempt. If the stream doesn't recover in this amount of time, that recovery attempt will be marked as failed, and the SDK will proceed with another attempt (if any left)
- **recovery_backoff_ms** (int, default: **2000**): The time in milliseconds to wait between recovery attempts.

Example: If the client specifies the following parameters:

- `recovery=True`
- `recovery_retries=4`
- `recovery_timeout_ms=5000`
- `recovery_backoff_ms=1000`

These parameters mean that the SDK will try to recover from the failure, and if it fails to recover in 4 attempts, it will throw an error (`ZerobusException`). If a single attempt doesn't finish in 5 seconds, SDK will mark it as failed and go on to the next one, until it runs out of attempts. Between each attempt, SDK will make 1 second "break".

Error handling

The only exception the SDK can throw is `ZerobusException`. This can be raised from any stream method (`create_stream`, `ingest_record`, `flush`, `close`). Once this exception is raised, the stream has failed permanently and cannot be reused.

Even with recovery enabled, an `ZerobusException` can be thrown if the SDK fails to recover after all retry attempts. This typically indicates a more permanent issue (e.g., persistent network loss, invalid credentials). When this happens, the client is responsible for handling the failure.

The `ZerobusStream` object provides a method, `get_unacked_records()`, which returns an iterator over all records that were sent but not yet acknowledged by the server. You can use this to preserve data when a stream fails.

Example Error Handling (Asynchronous Client)

Below is an example of how you might handle errors when automatic recovery is disabled. The logic is similar for the sync client, but would not use `async/await`.

```
Python
import asyncio
import row_pb2
from zerobus_sdk import ZerobusSdk, TableProperties,
StreamConfigurationOptions, ZerobusException, get_zerobus_token

DATABRICKS_WORKSPACE_URL = "<YOUR_DATABRICKS_WORKSPACE_URL>"
TABLE_NAME = "unity.default.air_quality"
CLIENT_ID = "<CLIENT_ID>"
CLIENT_SECRET = "<CLIENT_SECRET>"
```

```

SERVER_ENDPOINT = "<WORKSPACE_ID>.zerobus.<region>.cloud.databricks.com"

sdk_handle = ZerobusSdk(SERVER_ENDPOINT)

table_properties = TableProperties(TABLE_NAME, row_pb2.AirQuality.DESRIPTOR)
options = StreamConfigurationOptions(
    recovery=False,
    token_factory=lambda: get_zerobus_token(
        TABLE_NAME, SERVER_ENDPOINT.split(".")[0], DATABRICK_WORKSPACE_URL,
        CLIENT_ID, CLIENT_SECRET))

def preserve_unacked_records(unacked_records):
    # Customer logic to handle unacked_records
    pass

# Customer configuration (here just in sake of demonstration)
client_config = {'exitOnStreamFailure': True}

async def example_error_handling_without_recovery():
    try:
        stream = await sdk_handle.create_stream(table_properties, options)
    except ZerobusException as e:
        # Handle stream creation failure
        exit(1)

    # ... successful records ingestion ....

    try:
        await stream.ingest_record(row_pb2.AirQuality(
            device_name="device_num",
            temp=11,
            humidity=11
        ))
    except ZerobusException as e:
        # Handle stream failure options:
        if client_config['exitOnStreamFailure']:
            unacked_records = stream.get_unacked_records()
            preserve_unacked_records(unacked_records)
            exit(1)
        else:
            stream = await sdk_handle.recreate_stream(stream)
            # stream recovered, all unacked records will be re-ingested
            # no further actions for client

```

```

# ... ingesting ...

# End: Wait for all acks to be received.
try:
    await stream.flush()
except ZerobusException as e:
    # Flush failed -> same recovery as for ingesting
    # option 1 -> preserve unacked records and do something specific
    # option 2 -> recreate stream and then call flush()

if __name__ == "__main__":
    asyncio.run(example_error_handling_without_recovery())

```

Note: Error handling by the SDK will handle recovery for temporary failures for you, and users won't see it at all, so *ZerobusException* won't be raised so often. Once *ZerobusException* is raised, that means that SDK didn't manage to recover after a specified amount of time and retries, so users still have to handle it on their own. This should be a very rare situation, usually signals some permanent (non-retriable) errors. For example, network connection loss.

Java SDK Patterns

Blocking Call

```

Java
package com.databricks.example;

import com.databricks.zerobus_sdk.ZerobusSdk;
import com.databricks.zerobus_sdk.ZerobusStream;
import com.databricks.zerobus_sdk.TableProperties;
import com.databricks.zerobus_sdk.ZerobusException;
import com.databricks.zerobus_sdk.IngestRecordResult;
import com.databricks.zerobus_sdk.StreamConfigurationOptions;
import com.databricks.zerobus.IngestRecordResponse;
import com.databricks.test.table.TestTableRow.ClickBenchRow;
import java.util.concurrent.CompletableFuture;
import java.util.function.Consumer;
import java.util.ArrayList;
import java.util.concurrent.ExecutionException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.util.concurrent.CompletionException;

public class SampleClientBlocking {
    private static final Logger logger = LoggerFactory.getLogger(SampleClientBlocking.class);

```

```

private static final String TABLE_NAME = "unity.catalog.air_quality";
private static final String ZEROBUS_ENDPOINT = "<WORKSPACE_ID>.ingest.cloud.databricks.com";
private static final String UNITY_CATALOG_ENDPOINT = "<YOUR_DATABRICKS_WORKSPACE_URL>";
private static final String UC_TOKEN = "<YOUR_PAT>";

private static final int NUM_RECORDS = 10;

public static void main(String[] args) {
    try {
        Consumer<IngestRecordResponse> ackCallback = (response) -> {
            logger.info("Record ack for offset: {}", response.getDurabilityAckUpToOffset());
        };

        TableProperties<RecordOuterClass.AirQuality> tableProperties = new
TableProperties<RecordOuterClass.AirQuality>(TABLE_NAME, RecordOuterClass.AirQuality.class);
        StreamConfigurationOptions options = StreamConfigurationOptions.builder()
            .setMaxInflightRecords(100_000)
            .setAckCallback(ackCallback)
            .build();

        ZerobusSdk zerobusSdk = new ZerobusSdk(ZEROBUS_ENDPOINT, UNITY_CATALOG_ENDPOINT,
UC_TOKEN);

        ZerobusStream<RecordOuterClass.AirQuality> stream =
zerobusSdk.<RecordOuterClass.AirQuality>createStream(tableProperties, options);

        RecordOuterClass.AirQuality row = RecordOuterClass.AirQuality.newBuilder()
            .setDeviceName("device_1")
            .setTemp(20)
            .setHumidity(50)
            .build();

        for (int i = 0; i < NUM_RECORDS; i++) {
            IngestRecordResult result = stream.ingestRecord(row);
            result.getRecordAccepted().get();
            result.getWriteCompleted().get();
        }

        stream.close();
    } catch (ZerobusException | InterruptedException | ExecutionException e) {
        logger.error("Error in stream: {}", e.getMessage(), e);
    }
}
}

```

Non-Blocking Call

```

Java
package com.databricks.example;

```

```

import com.databricks.zerobus_sdk.ZerobusSdk;
import com.databricks.zerobus_sdk.ZerobusStream;
import com.databricks.zerobus_sdk.TableProperties;
import com.databricks.zerobus_sdk.ZerobusException;
import com.databricks.zerobus_sdk.IngestRecordResult;
import com.databricks.zerobus_sdk.StreamConfigurationOptions;
import com.databricks.zerobus.IngestRecordResponse;
import com.databricks.test.table.TestTableRow.ClickBenchRow;
import java.util.concurrent.CompletableFuture;
import java.util.function.Consumer;
import java.util.ArrayList;
import java.util.concurrent.ExecutionException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.util.concurrent.CompletionException;

public class SampleClientBlocking {
    ArrayList<CompletableFuture> futures = new ArrayList<>();

    private static final Logger logger = LoggerFactory.getLogger(SampleClientBlocking.class);

    private static final String TABLE_NAME = "unity.catalog.air_quality";
    private static final String ZEROBUS_ENDPOINT = "<WORKSPACE_ID>.ingest.cloud.databricks.com";
    private static final String UNITY_CATALOG_ENDPOINT = "<YOUR_DATABRICKS_WORKSPACE_URL>";
    private static final String UC_TOKEN = "<YOUR_PAT>";

    private static final int NUM_RECORDS = 10;

    public static void main(String[] args) {
        try {
            Consumer<IngestRecordResponse> ackCallback = (response) -> {
                logger.info("Record ack for offset: {}", response.getDurabilityAckUpToOffset());
            };

            TableProperties<RecordOuterClass.AirQuality> tableProperties = new
TableProperties<RecordOuterClass.AirQuality>(TABLE_NAME, RecordOuterClass.AirQuality.class);
            StreamConfigurationOptions options = StreamConfigurationOptions.builder()
                .setMaxInflightRecords(100_000)
                .setAckCallback(ackCallback)
                .build();

            ZerobusSdk zerobusSdk = new ZerobusSdk(ZEROBUS_ENDPOINT, UNITY_CATALOG_ENDPOINT,
UC_TOKEN);

            ZerobusStream<RecordOuterClass.AirQuality> stream =
zerobusSdk.<RecordOuterClass.AirQuality>createStream(tableProperties, options);

            RecordOuterClass.AirQuality row = RecordOuterClass.AirQuality.newBuilder()
                .setDeviceName("device_1")
                .setTemp(20)
                .setHumidity(50)
                .build();

            for (int i = 0; i < NUM_RECORDS; i++) {
                IngestRecordResult result = stream.ingestRecord(row);
                result.getRecordAccepted().get();
                futures.add(result.getWriteCompleted());
            }
        }
    }
}

```

```

    }

    stream.flush();

    stream.close();
} catch (ZeroBusException | InterruptedException | ExecutionException e) {
    logger.error("Error in stream: {}", e.getMessage(), e);
} finally {
    int failed = 0;
    for (CompletableFuture future : futures) {
        try {
            future.get();
        } catch (CompletionException | InterruptedException | ExecutionException e) {
            failed++;
        }
    }

    logger.info("Number of failed ingest records: {}", failed);
}
}
}

```

Python SDK

This section provides a more detailed look at the SDK's classes and methods.

Note: The default import `from zerobus_sdk import ...` provides the **synchronous** client. For the asynchronous client, import from `zerobus_sdk. aio`. The signatures below are for the synchronous client; the async client uses `async def` for I/O methods and returns `asyncio.Future` where noted.

TableProperties Class

```

Python
class TableProperties:
    """
    Table properties for the stream, describing the table stream will ingest
    data to.
    """
    def __init__(self, table_name: str, descriptor_proto: Descriptor):
        """
        Args:
            table_name (str): Three-part qualified name represented by

```

```

        <catalog>.<database>.<table>.
        descriptor_proto (google.protobuf.descriptor.Descriptor): Protobuf
message
        description that will be used to encode
messages.
    """
    pass

```

StreamConfigurationOptions Class

```

Python
class StreamConfigurationOptions:
    """
    Configuration options for the stream.
    """
    def __init__(self, **kwargs):
        """
        Token factory is a required argument in order for the SDK to work
properly.
        Other args are optional, with default values below.

        Args:
            max_inflight_records (int) DEFAULT = 50_000
                Maximum number of records that SDK can accept and
                send to the server and wait for them to be acknowledged.
                For records ingested after number of inflight records reached
                this threshold, ingest_record will be blocked until some
records
                are acked by the server.

            recovery (bool) DEFAULT = True
                Whether to enable automatic recovery of the stream in case
                of stream failure.

            recovery_retries (int) DEFAULT = 3
                Number of attempts SDK will do during recovery.

            recovery_timeout_ms (int) DEFAULT = 7000
                Timeout for stream recovery in milliseconds (for one attempt).

            recovery_backoff_ms (int) DEFAULT = 2000
                Backoff time in milliseconds between recovery attempts.

```

```

server_lack_of_ack_timeout_ms (int) DEFAULT = 60_000
    The number of ms in which, if we do not receive an
    acknowledgement from the server, the server will be
    considered unresponsive.

flush_timeout_ms (int) DEFAULT = 300_000
    Timeout for flushing the stream in milliseconds.

ack_callback (func(IncomingRecordResponse)): Optional DEFAULT = None
    Callback function to run on each "ack" received from the
server.
    Note that the server will return the largest offset it
acknowledged.

token_factory (func() -> str): Required DEFAULT = None
    Function that is called each time a stream is created to fetch
the token
    used for authentication
"""
pass

```

ZerobusException

```

Python
class ZerobusException(Exception):
    """
    The only exception the SDK will throw. Once it's thrown, the stream has
    failed permanently and can't be reused.
    """
    def __init__(self, message: str):
        super().__init__(message)

```

ZerobusSdk

```

Python
class ZerobusSdk():
    def __init__(self, host: str):
        """
        Creates a Python Ingest SDK client, which wraps gRPC functions.

```

```

    Args:
        host (str): Ingestion API server endpoint.
    """
    pass

def create_stream(
    self,
    table_properties: TableProperties,
    options: StreamConfigurationOptions = StreamConfigurationOptions()
) -> ZerobusStream:
    """
    Creates a gRPC stream to ingest data to the table.

    This opens a stream which can be used for blocking and/or non-blocking
    gRPC calls to ingest data to the given table. At this stage, the table
    and message descriptor will be validated.

    Args:
        table_properties (TableProperties): Describes the target table.
        options (StreamConfigurationOptions): Stream configuration options.

    Returns:
        ZerobusStream: An object representing the stream.
    """
    pass

def recreate_stream(self, old_stream: ZerobusStream) -> ZerobusStream:
    """
    Recreates a stream from a previously failed stream.
    It uses the same table_properties and stream configuration options.
    All unacknowledged records from the old stream are re-ingested.

    Args:
        old_stream (ZerobusStream): The failed stream.

    Returns:
        ZerobusStream: A new, recovered stream.
    """
    pass

```

ZerobusStream

Python

```
class ZerobusStream:
    """
    Class representing a stream. Customers use this class to ingest records.
    It can't be created directly, but only via ZerobusSdk.create_stream().
    """

    def ingest_record(self, record) -> RecordAcknowledgment:
        """
        Queues a record for sending. In the sync client, this returns a
        `RecordAcknowledgment` object. In the async client, this is an
        `async` method that returns an `asyncio.Future`.

        Args:
            record (Message): A Protobuf message with a descriptor matching
                             the one passed to TableProperties.

        Returns:
            RecordAcknowledgment: A future-like object to wait for the ack.
        """
        pass

    def get_unacked_records(self) -> Iterator[record]:
        """
        Once a stream has failed, this method will return an iterator
        for all records that were not durable / acked.
        """
        pass

    def flush(self):
        """
        Waits for all queued records to be durable and acked by the server.
        The stream can be used after flush() completes.
        """
        pass

    def close(self):
        """
        Closes the stream.
        It first calls flush(), ensuring that all records are durable.
        This method is idempotent. Once a stream is closed it can't be reused.
        """
        pass
```

RecordAcknowledgment - sync specific

Python

```
class RecordAcknowledgment:
    """
    A thread-safe, future-like object for blocking until a record is
    acknowledged.
    (Only used in the synchronous client).
    """

    def wait_for_ack(self, timeout_sec: Optional[float] = None) -> int:
        """
        Blocks until the acknowledgment is received or an optional timeout
        occurs.

        Args:
            timeout_sec (Optional[float]): The maximum number of seconds to
            wait.

        Returns:
            int: The offset id of the acknowledgment.

        Raises:
            ZerobusException: If the wait times out or the operation failed.
        """
        pass

    def add_done_callback(self, callback: Callable[[int], None]):
        """
        Adds a per-record callback to be executed upon acknowledgment.
        If the ack has already been received, the callback is executed
        immediately.
        """
        pass

    def is_done(self) -> bool:
        """
        Checks if the operation has completed (i.e., ack has been received).
        """
        return False
```

Rust SDK

See below for a more detailed look at the SDK.

TableProperties struct

```
Rust
/// Table properties for the stream, describing the table stream will ingest data
to.
pub struct TableProperties {
    /// Three-part qualified name with represented by
    <catalog>.<database>.<table>.
    pub table_name: String,
    /// Protobuf messages description that will be used to encode the messages.
    pub descriptor_proto: prost_types::DescriptorProto,
}
```

StreamConfigurationOptions struct

```
Rust
/// Configuration options for the stream
pub struct StreamConfigurationOptions {
    /// Maximum number of records that SDK can accept and
    /// send to the server and wait for them to be acknowledged.
    pub max_inflight_records: usize,
}

impl Default for StreamConfigurationOptions {
    fn default() -> Self {
        Self {
            max_inflight_records: 50_000,
        }
    }
}
```

StreamType enum

Rust

```
/// Stream type
pub enum StreamType {
    /// Ephemeral streams exist only for the duration of the connection.
    /// They are not persisted and are not recoverable.
    Ephemeral,
    /// Persistent streams are durable and recoverable.
    Persistent,
}
```

ZerobusError enum

Rust

```
/// Error types
pub enum ZerobusError {
    #[error("Failed to open a channel: {0}.")]
    ChannelCreationError(String),
    #[error("Failed to create stream: {0}.")]
    CreateStreamError(tonic::Status),
    #[error("Failed to establish TLS connection.")]
    FailedToEstablishTlsConnectionError,
    #[error("The specified Zerobus endpoint is in invalid format: {0}.")]
    InvalidZerobusEndpointError(String),
    #[error("Specified UC table is invalid: {0}.")]
    InvalidTableName(String),
    #[error("Specified UC endpoint is in invalid format: {0}.")]
    InvalidUCEndpointError(String),
    #[error("Specified UC token is in invalid format: {0}.")]
    InvalidUCTokenError(String),
    #[error("Stream is closed: {0}")]
    StreamClosedError(tonic::Status),
    #[error("Invalid argument: {0}.")]
    InvalidArgument(String),
    #[error("Unexpected response from server. Response: {0}")]
}
```

```
UnexpectedStreamResponseError(String),  
}
```

ZerobusSdk struct

Rust

/// Ingest SDK, which wraps gRPC functions

```
pub struct ZerobusSdk {  
    /// Ingestion API server endpoint  
    pub zerobus_endpoint: String,  
    /// Ingestion URI  
    pub unity_catalog_url: Option<String>,  
    /// Databricks personal access token  
    pub token: Option<String>,  
}
```

```
impl ZerobusSdk {  
    pub fn new(  
        zerobus_endpoint: String,  
        unity_catalog_url: Option<String>,  
        token: Option<String>,  
    ) -> Self {  
        ZerobusSdk {  
            zerobus_endpoint,  
            unity_catalog_url,  
            token,  
        }  
    }  
}
```

/// Create a gRPC stream to ingest data to the table. For now, we are opening a new channel for each stream.

```
#[instrument(level = "debug", skip(self, table_properties, options))]  
pub async fn create_stream(  
    &self,
```

```

        table_properties: TableProperties,
        options: Option<StreamConfigurationOptions>,
    ) -> Result<ZerobusStream, ZerobusError> {
        /// Arguments:
        /// table_properties (Tableproperties) - Parameter that describes a table
stream should be ingesting to.
        /// options (Option<StreamConfigurationOptions>) - Stream configuration
options.
        /// It's optional, and if not given, steam will be created with default
values.
        /// See StreamConfigurationOptions

        /// Returns:
        /// ZerobusStream, ZerobusError - Representing the stream and error code.
    }
}

```

ZerobusStream struct

Rust

```

pub type ZerobusResult<T> = Result<T, ZerobusError>;

/// Struct representing Stream.
/// Customers use this class to ingest records and stream data.
/// It can't be created directly, but using ZerobusSdk::create_stream.
pub struct ZerobusStream {
    pub stream_id: String,
    /// Type of stream, currently only supports Ephemeral streams
    pub stream_type: StreamType,
    /// The Unity Catalog endpoint
    pub unity_catalog_url: Option<String>,
    /// The Unity Catalog token.
    pub token: Option<String>,
}

```

```

    /// Configuration options
    pub options: StreamConfigurationOptions,
}

impl ZerobusStream {
    /// Ingests a record into the stream.
    /// Returns a future that resolves to the offset id of the ingested record.
    pub async fn ingest_record(&self,
        payload: Vec<u8>,
    ) -> ZerobusResult<impl Future<Output = ZerobusResult<i64>>> {
        /// Arguments:
        /// payload (Vec<u8>) - Proto encoded record

        /// Returns:
        /// ZerobusResult - Representing a Future (sdk::future::Future) and error
        code.

    }

    /// Flushes the stream up to the latest sent offset.
    pub async fn flush(&self) -> Result<(), ZerobusError> {
        /// Returns:
        /// Result<(), ZerobusError> - Representing a Future (sdk::future::Future)
        that resolves when the stream is caught up to the given offset and error code.

        /// Note that stream can continue to ingest while flush is in progress.
        The flush
        /// request will capture the state of the stream at the time of the
        request and
        /// will not wait for records that are ingested while the flush is in
        progress.

    }

    /// Closes the stream with flushing all pending records first.
    pub async fn close(&self) -> Result<(), ZerobusError> {

```

```

    }
}

```

Java SDK

See below for a more detailed look at the SDK.

TableProperties Class

```

Java
/**
 * Table properties for the stream, describes the table to ingest records into.
 *
 * @param tableName The name of the table to ingest records into.
 * @param recordType The type of records to be ingested.
 */
case class TableProperties[RecordType <: ProtoMessage[RecordType]](
  tableName: String,
  recordType: Class[RecordType]
);

```

StreamConfigurationOptions Class

```

Java
/**
 * Configuration options for Zerobus streams.
 *
 * This class provides various settings to control stream behavior including
 * performance tuning, error handling, and callback configuration.
 *
 * Use the builder pattern to create instances:
 *
 * StreamConfigurationOptions options = StreamConfigurationOptions.builder()
 *   .setMaxInflightRecords(50000)
 *   .setRecovery(true)
 *   .setAckCallback(response ->
 *     System.out.println("Acked: " + response.getDurabilityAckUpToOffset()))
 *   .build();
 */
public class StreamConfigurationOptions {

  /**
   * A simple wrapper for optional values.
   *

```

```

    * This class provides a way to represent optional values in a type-safe manner,
    * similar to Scala's Option type.
    *
    * @param <T> the type of the wrapped value
    */
    public static class Option<T> {
        /**
         * Creates an Option containing the specified value.
         *
         * @param value the value to wrap
         */
        public Option(T value);

        /**
         * Creates an empty Option (no value).
         */
        public Option();

        /**
         * Returns the wrapped value.
         *
         * @return the wrapped value, or null if this Option is empty
         */
        public T get();

        /**
         * Checks if this Option contains a value.
         *
         * @return true if this Option contains a value, false otherwise
         */
        public boolean isDefined();
    };

    /**
     * Returns the maximum number of records that can be in flight.
     *
     * This controls how many records the SDK can accept and send to the server
     * before waiting for acknowledgments. Higher values improve throughput but
     * use more memory.
     *
     * @return the maximum number of in-flight records
     */
    public int maxInflightRecords();

    /**
     * Returns whether automatic recovery is enabled.
     *
     * When enabled, the SDK will automatically attempt to recover from stream
     * failures by recreating the stream and resending unacknowledged records.
     *
     * @return true if automatic recovery is enabled, false otherwise
     */
    public boolean recovery();

    /**
     * Returns the timeout for recovery operations.
     *

```

```

    * This is the maximum time to wait for a recovery operation to complete
    * before considering it failed.
    *
    * @return the recovery timeout in milliseconds
    */
    public int recoveryTimeoutMs();

    /**
     * Returns the backoff delay between recovery attempts.
     *
     * This is the delay to wait between consecutive recovery attempts when
     * automatic recovery is enabled.
     *
     * @return the recovery backoff delay in milliseconds
     */
    public int recoveryBackoffMs();

    /**
     * Returns the maximum number of recovery attempts.
     *
     * This is the maximum number of times the SDK will attempt to recover
     * from a stream failure before giving up.
     *
     * @return the maximum number of recovery attempts
     */
    public int recoveryRetries();

    /**
     * Returns the timeout for flush operations.
     *
     * This is the maximum time to wait for a flush operation to complete
     * before considering it failed.
     *
     * @return the flush timeout in milliseconds
     */
    public int flushTimeoutMs();

    /**
     * Returns the timeout for server acknowledgment.
     *
     * This is the maximum time to wait for the server to acknowledge records
     * before considering the server unresponsive.
     *
     * @return the server acknowledgment timeout in milliseconds
     */
    public int serverLackOfAckTimeoutMs();

    /**
     * Returns the acknowledgment callback function.
     *
     * This callback is invoked whenever the server acknowledges records.
     * If no callback is set, this returns an empty Option.
     *
     * NOTE: It is possible that callbacks are called after flush finishes.
     *
     * @return the acknowledgment callback, or an empty Option if none is set
     */

```

```

public Option<Consumer<IngestRecordResponse>> ackCallback();

/**
 * Returns the default stream configuration options.
 *
 * Default values:
 * - maxInflightRecords: 50000
 * - recovery: true
 * - recoveryTimeoutMs: 15000
 * - recoveryBackoffMs: 2000
 * - recoveryRetries: 3
 * - flushTimeoutMs: 300000
 * - serverLackOfAckTimeoutMs: 60000
 * - ackCallback: empty
 *
 * @return the default stream configuration options
 */
public static StreamConfigurationOptions getDefault();

/**
 * Returns a new builder for creating StreamConfigurationOptions.
 *
 * @return a new StreamConfigurationOptionsBuilder
 */
public static StreamConfigurationOptionsBuilder builder();

/**
 * Builder for creating StreamConfigurationOptions instances.
 *
 * This builder provides a fluent API for configuring stream options.
 * All parameters have sensible defaults if not specified.
 *
 * Example usage:
 *
 * StreamConfigurationOptions options = StreamConfigurationOptions.builder()
 *     .setMaxInflightRecords(100000)
 *     .setRecovery(false)
 *     .setAckCallback(response ->
 *         System.out.println("Record acked: " + response.getDurabilityAckUpToOffset()))
 *     .build();
 *
 * @see StreamConfigurationOptions
 * @since 1.0.0
 */
public static class StreamConfigurationOptionsBuilder {

    /**
     * Sets the maximum number of records that can be in flight.
     *
     * This controls how many records the SDK can accept and send to the server
     * before waiting for acknowledgments. Higher values improve throughput but
     * use more memory.
     *
     * @param maxInflightRecords the maximum number of in-flight records
     * @return this builder for method chaining
     */
    public StreamConfigurationOptionsBuilder setMaxInflightRecords(int maxInflightRecords);

```

```

/**
 * Sets whether automatic recovery is enabled.
 *
 * When enabled, the SDK will automatically attempt to recover from stream
 * failures by recreating the stream and resending unacknowledged records.
 *
 * @param recovery true to enable automatic recovery, false to disable
 * @return this builder for method chaining
 */
public StreamConfigurationOptionsBuilder setRecovery(boolean recovery);

/**
 * Sets the timeout for recovery operations.
 *
 * This is the maximum time to wait for a recovery operation to complete
 * before considering it failed.
 *
 * @param recoveryTimeoutMs the recovery timeout in milliseconds
 * @return this builder for method chaining
 */
public StreamConfigurationOptionsBuilder setRecoveryTimeoutMs(int recoveryTimeoutMs);

/**
 * Sets the backoff delay between recovery attempts.
 *
 * This is the delay to wait between consecutive recovery attempts when
 * automatic recovery is enabled.
 *
 * @param recoveryBackoffMs the recovery backoff delay in milliseconds
 * @return this builder for method chaining
 * @throws IllegalArgumentException if recoveryBackoffMs is less than 0
 */
public StreamConfigurationOptionsBuilder setRecoveryBackoffMs(int recoveryBackoffMs);

/**
 * Sets the maximum number of recovery attempts.
 *
 * This is the maximum number of times the SDK will attempt to recover
 * from a stream failure before giving up.
 *
 * @param recoveryRetries the maximum number of recovery attempts
 * @return this builder for method chaining
 */
public StreamConfigurationOptionsBuilder setRecoveryRetries(int recoveryRetries);

/**
 * Sets the timeout for flush operations.
 *
 * This is the maximum time to wait for a flush operation to complete
 * before considering it failed.
 *
 * @param flushTimeoutMs the flush timeout in milliseconds
 * @return this builder for method chaining
 */
public StreamConfigurationOptionsBuilder setFlushTimeoutMs(int flushTimeoutMs);

```

```

    /**
     * Sets the timeout for server acknowledgment.
     *
     * This is the maximum time to wait for the server to acknowledge records
     * before considering the server unresponsive.
     *
     * @param serverLackOfAckTimeoutMs the server acknowledgment timeout in milliseconds
     * @return this builder for method chaining
     */
    public StreamConfigurationOptionsBuilder setServerLackOfAckTimeoutMs(int
serverLackOfAckTimeoutMs);

    /**
     * Sets the acknowledgment callback function.
     *
     * This callback is invoked whenever the server acknowledges records.
     * The callback receives an IngestRecordResponse containing information
     * about the acknowledged records.
     *
     * @param ackCallback the acknowledgment callback function
     * @return this builder for method chaining
     */
    public StreamConfigurationOptionsBuilder setAckCallback(Consumer<IngestRecordResponse>
ackCallback);

    /**
     * Builds a new StreamConfigurationOptions instance.
     *
     * @return a new StreamConfigurationOptions with the configured settings
     */
    public StreamConfigurationOptions build();
};
}

```

ZerobusException Class

```

Java
/**
 * Base exception class for all Zerobus related errors.
 * This allows clients to catch all Zerobus specific exceptions with a single catch block.
 */
class ZerobusException(
    message: String,
    cause: Throwable = null
) extends Exception(message, cause)

/**
 * An exception that indicates a non-retriable error has occurred.
 * This is used to signal that stream creation or recovery should not be retried.
 */
class NonRetriableException(
    message: String,
    cause: Throwable = null

```

```
) extends ZerobusException(message, cause)
```

ZerobusSdk Class

Java

```
/**
 * The main entry point for the Zerobus SDK.
 *
 * This class provides methods to create and recreate streams for ingesting records
 * into Databricks tables. It handles authentication, connection management,
 * and stream lifecycle operations.
 */
public class ZerobusSdk extends CommonZerobusSdk {

    /**
     * Creates a new ZerobusSdk instance.
     *
     * @param endpoint The gRPC endpoint URL for the Zerobus service.
     * @param unityCatalogEndpoint The Unity Catalog endpoint URL.
     * @param token The authentication token for Unity Catalog.
     */
    public ZerobusSdk(String endpoint, String unityCatalogEndpoint, String token);

    /**
     * Creates a new gRPC stream for ingesting records into a table.
     *
     * Opens a stream which can be used for blocking and/or non-blocking gRPC calls
     * to ingest data to the given table.
     * At this stage, the table and message descriptor will be validated.
     *
     * @param tableProperties Configuration for the target table including table name
     * and record type information.
     * @param options Configuration options for the stream including timeouts,
     * retry settings, and callback functions.
     * @param RecordType The type of records to be ingested (must extend ProtoMessage).
     * @return A Future that completes with an ZerobusStream when the stream is ready.
     *
     * @throws ZerobusException If creating the stream fails.
     */
    public <RecordType extends com.google.protobuf.Message> ZerobusStream<RecordType>
    createStream(TableProperties<RecordType> tableProperties, StreamConfigurationOptions options) throws
    ZerobusException;

    public <RecordType extends com.google.protobuf.Message> ZerobusStream<RecordType>
    createStream(TableProperties<RecordType> tableProperties) throws ZerobusException;

    /**
     * Recreate stream from a failed stream.
     *
     * Uses the same table properties and stream options as the failed stream.
     * It will also ingest all unacknowledged records from the failed stream.
     */
}
```

```

    *
    * @param stream The stream to be recreated.
    * @tparam RecordType The type of records to be ingested (must extend ProtoMessage).
    * @return A Future that completes with an ZerobusStream when the stream is ready.
    *
    * @throws ZerobusException If recreating the stream fails.
    */
    public <RecordType extends com.google.protobuf.Message> ZerobusStream<RecordType>
    recreateStream(ZerobusStream<RecordType> stream) throws ZerobusException;
}

```

IngestRecordResult Class

Java

```

/**
 * Represents the result of ingesting a record.
 *
 * @param recordAccepted A CompletableFuture that completes when the SDK accepts the record for
 * processing
 * @param writeCompleted A CompletableFuture that completes when the record has been successfully
 * written to storage
 */
public class IngestRecordResult {
    private final CompletableFuture<Void> recordAccepted;
    private final CompletableFuture<Void> writeCompleted;

    /**
     * Creates a new IngestRecordResult with the specified futures.
     *
     * @param recordAccepted A CompletableFuture that completes when the SDK accepts the record for
     * processing
     * @param writeCompleted A CompletableFuture that completes when the record has been successfully
     * written to storage
     */
    public IngestRecordResult(CompletableFuture<Void> recordAccepted, CompletableFuture<Void>
    writeCompleted) {
        this.recordAccepted = recordAccepted;
        this.writeCompleted = writeCompleted;
    }

    /**
     * Returns the CompletableFuture that completes when the SDK accepts the record for processing.
     *
     * @return The CompletableFuture for record acceptance
     */
    public CompletableFuture<Void> getRecordAccepted() {
        return recordAccepted;
    }

    /**
     * Returns the CompletableFuture that completes when the record has been successfully written to
     * storage.
     *

```

```

    * @return The CompletableFuture for write completion
    */
    public CompletableFuture<Void> getWriteCompleted() {
        return writeCompleted;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        IngestRecordResult that = (IngestRecordResult) obj;
        return Objects.equals(recordAccepted, that.recordAccepted) &&
            Objects.equals(writeCompleted, that.writeCompleted);
    }

    @Override
    public int hashCode() {
        return Objects.hash(recordAccepted, writeCompleted);
    }

    @Override
    public String toString() {
        return "IngestRecordResult{" +
            "recordAccepted=" + recordAccepted +
            ", writeCompleted=" + writeCompleted +
            '}';
    }
}

```

ZerobusStream Class

```

Java
/**
 * Zerobus stream for ingesting records into a table.
 * Should be created using ZerobusSdk.createStream.
 */
public class ZerobusStream<RecordType extends ProtoMessage<RecordType>> {

    /**
     * Returns the ID of the stream.
     *
     * @return The ID of the stream.
     */
    public String getStreamId();

    /**
     * Returns the state of the stream.
     *
     * @return The state of the stream.
     */
    public StreamState getState();

    /**

```

```

    * Returns the unacknowledged records after stream failure.
    *
    * @return The unacknowledged records after stream failure.
    */
    public RecordIterator<RecordType> getUnackedRecords();

    /**
     * Ingests a record into the stream.
     *
     * @param record The record to ingest.
     * @return A CompletableFuture that will be completed when the SDK accepts the record.
     * The future contains another CompletableFuture that will be completed when the server
    acknowledges the record.
     * If the future raises an exception, the record most probably was not acknowledged,
     * but it is also possible that the server acknowledged the record but the response was lost.
     * In this case client should decide whether to retry the record or not.
     */
    public IngestRecordResult ingestRecord(RecordType record);

    /**
     * Flushes the stream, waiting for all queued records to be acknowledged by the server.
     * The stream doesn't close after flushing.
     *
     * NOTE: It is possible that callbacks are called after flush finishes.
     *
     * @throws ZerobusException If the stream is not opened.
     */
    public void flush() throws ZerobusException;

    /**
     * Closes the stream, while first flushing all queued records.
     * Once a stream is closed, it cannot be reopened.
     *
     * @throws ZerobusException If the stream is not opened.
     */
    public void close() throws ZerobusException;
}

```